SoftwareX 34 (2026) 102550

Contents lists available at ScienceDirect

SoftwareX

journal homepage: www.elsevier.com/locate/softx

GreenAccounter: A toolkit for carbon-aware orchestration of deep learning workloads in geo-distributed clouds

Jeonghyeon Park, Jaekyeong Kim, Wonseok Son, Sejin Chun*

Department of Computer Engineering, Dong-A University, Busan, Republic of Korea

HIGHLIGHTS

- Implements carbon-aware orchestration for deep learning in multi-cloud environments.
- Enables checkpoint-based migration while preserving training states and continuity.
- Provides real-time monitoring of carbon intensity and energy metrics.
- Offers an open-source platform for reproducible, emissions-aware research.

ARTICLE INFO

Keywords:
Carbon-aware
Deep learning
Workload migration
Multi-cloud environments

ABSTRACT

Deep learning workloads generate substantial carbon emissions in data centers, largely because training is both time-consuming and energy-intensive. To address this challenge, previous studies have explored either temporal workload shifting or spatial workload migration. Still, these approaches remain limited for long-running workloads, such as Large Language Models (LLMs), because they fail to adapt to continuous fluctuations in regional carbon intensity. In this paper, we introduce **GreenAccounter**, a toolkit for carbon-aware orchestration of deep learning workloads across multi-cloud environments. It integrates real-time carbon intensity monitoring with checkpoint-based migration, allowing training to continue seamlessly while reducing emissions. A unified dashboard visualizes regional carbon intensity, cumulative emissions, and power consumption, providing operators with a single pane of glass for managing distributed cloud resources. **GreenAccounter** serves as both (i) a reproducible research platform for carbon-aware scheduling and (ii) a practical operational toolkit for emissions reduction in AI training. As an open-source release, it promotes sustainable, transparent, and data-driven practices for large-scale deep learning.

Metadata

Code metadata (mandatory)

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/datascience-labs/GreenAccounter
C3	Permanent link to Reproducible Capsule	https://zenodo.org/records/18085272
C4	Legal Code License	MIT
C5	Code versioning system used	Git / GitHub Actions
C6	Software code languages, tools, and services used	Python, React, Flask Nginx, Docker Kubernetes Grafana, Jaeger, Kiali Electricity Maps API Firebase Realtime Database Firebase Storage
C7	Compilation requirements, operating environments & dependencies	The provided link(C2 and C3) is for a demo version; the full toolkit is designed and optimized for deployment on the Naver Cloud Platform
C8	If available Link to developer documentation/manual	Included in README.md file
C9	Support email for questions	jeonghyeon.park@datasciencelabs.org

* Corresponding author.
Email address: sjchun@dau.ac.kr (S. Chun).

https://doi.org/10.1016/j.softx.2026.102550
Received 23 October 2025; Received in revised form 29 December 2025; Accepted 1 February 2026
Available online 10 February 2026
2352-7110/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Motivation and significance

Artificial intelligence (AI) has rapidly advanced, driving innovation in science, healthcare, and industry. These breakthroughs rely heavily on training large-scale models with billions of parameters, which demand substantial computational and energy resources. For instance, training models such as GPT-4 requires massive power input due to their extreme computational complexity [1]. Consequently, carbon emissions from data centers have surged, now accounting for approximately 1% of global electricity consumption [2]. Furthermore, reports indicate that deep learning workloads represent nearly 15% of the total electricity consumed in Google's data centers [3]. This growing energy demand directly translates into significant carbon emissions during model training.

The rapid increase in deep learning workloads has become a global concern due to the accompanying rise in energy consumption and greenhouse gas emissions. Fundamentally, the issue is driven not merely by the expansion of computational resources, but by the carbon intensity of the underlying power grid. Carbon intensity denotes the amount of greenhouse gases emitted per kilowatt-hour (kWh) of electricity generated [4]. It fluctuates over time and across regions, depending on the energy mix—such as renewables and fossil fuels—as well as supply and demand conditions [5]. As a result, identical computations can produce drastically different carbon footprints depending on the time and location of execution.

Deep learning workloads typically run for extended durations and involve numerous iterative updates [6]. During training, models process mini-batches sequentially and update parameters incrementally over many epochs. Such processes often span several days or even weeks [3,4]. Interruptions in training lead to the loss of progress unless the current state is preserved. To address this, checkpointing is employed to periodically save model parameters and optimizer states, enabling seamless resumption after interruptions. Checkpointing is thus critical for ensuring continuity in long-term training [7]. However, this characteristic makes deep learning workloads difficult to migrate as isolated tasks under traditional scheduling approaches. Even during migration, the suspended training must be captured as a checkpoint and restored in the target environment to maintain training continuity [8].

To mitigate carbon emissions, prior research has proposed two main strategies: temporal workload shifting and spatial workload migration. Temporal shifting delays or advances workloads to periods of lower carbon intensity [9–15], while spatial migration relocates workloads to cloud regions with cleaner energy sources [6,8,14,16–20]. However, for long-running deep learning workloads, temporal adjustments alone cannot consistently avoid high-carbon periods due to the continuous variability of carbon intensity [21]. Likewise, spatial migration in traditional systems often fails to preserve the training, resulting in complete restarts and redundant computation [8].

Additionally, carbon monitoring dashboards offered by major cloud providers—such as Microsoft Azure, AWS, and Google Cloud—focus primarily on post-hoc reporting and aggregated usage statistics [22–24]. They lack mechanisms for active, real-time workload orchestration based on carbon intensity dynamics. Although recent frameworks have begun integrating real-time carbon data for scheduling decisions [25–27], these efforts generally target single or short-term workloads and remain inadequate for managing long-term, iterative processes such as deep learning workloads.

To address these limitations, we present **GreenAccounter**, an open-source toolkit for carbon-aware orchestration of deep learning workloads across geo-distributed cloud environments. Our core mechanism is carbon-aware checkpoint-based migration, which preserves training continuity while minimizing emissions. When a region's carbon intensity exceeds a predefined threshold, the current training state is checkpointed, and execution resumes in a lower-carbon region. Unlike static or reactive migration methods, this dynamic strategy adapts to real-time fluctuations in carbon intensity and reduces cumulative emissions over long-term training cycles.

The contributions of this study are as follows:

- **Carbon-aware migration for deep learning workloads:** GreenAccounter minimizes cumulative carbon emissions through a *checkpoint-based migration strategy* that dynamically relocates long-term training workloads to cloud regions with lower real-time carbon intensity.
- **Training continuity and state preservation:** During migration, GreenAccounter automatically generates checkpoints containing both model parameters and optimizer states. These checkpoints enable seamless restoration and uninterrupted training progress across heterogeneous cloud environments.
- **Unified multi-cloud observability:** GreenAccounter serves as a single platform that covers the entire operational cycle, including real-time measurement, optimization through migration, and comprehensive reporting. A unified dashboard provides a single pane of glass for monitoring regional carbon intensity and managing carbon-aware orchestration. It also visualizes cumulative emissions reports and energy consumption.
- **Cloud-native and extensible architecture:** GreenAccounter is implemented using a Kubernetes-based microservice architecture, ensuring scalability, resilience, and compatibility with major commercial cloud providers.

The remainder of this paper is organized as follows. Section 2 discusses the research context and related software. Section 3 describes the architecture and functionality of the proposed toolkit. Section 4 introduces the migration process and the developed interfaces. Section 5 discusses both research and practical implications. Finally, Section 6 presents the conclusion and future work.

2. Related work

In recent years, research on sustainable cloud computing has focused on strategies for reducing carbon emissions in cloud environments. These approaches propose mitigating emissions through temporal workload shifting [9–15] or spatial workload migration [6,8,14,16–20]. However, temporal shifting is limited for long-term workloads, as intervals frequently coincide with periods of high carbon emissions. Moreover, in regions with low variability in carbon intensity, time of execution adjustments provide only marginal reductions [21]. Spatial workload migration approaches also face fundamental limitations. They have primarily focused on workloads such as web services that do not require state preservation. Consequently, for workloads such as deep learning that run iteratively for extended periods and must maintain training state, preserving training state through simple spatial migration is difficult. Therefore, when existing scheduling algorithms are directly applied, training must restart from the beginning after cross-region migration. This restart incurs additional computation and data retransmission. As a result, cross-region transfer costs and latency constraints increase, raising the possibility that spatial migration of deep learning training workloads may lead to additional carbon emissions [16,21].

To execute workloads in cloud environments while considering energy or carbon emissions, existing studies have proposed energy consumption prediction [19,20] and carbon-aware scheduling [14,15], respectively. Raith et al. proposed a method to predict energy consumption based on container placement using Graph Neural Networks (GNNs) [20]. Saad et al. presented a method for training energy-consumption prediction models via federated learning in Kubernetes environments, enabling power estimation in a distributed setting without data sharing [19]. However, these methods focus on estimating energy consumption. They do not include mechanisms to continuously transfer deep learning training workloads or migrate them while maintaining the training state in response to temporal and spatial variations in carbon intensity. Piontek et al. proposed carbon-aware scheduling that extends the Kubernetes scheduler to perform workload placement based on carbon

intensity [15]. However, this method targets single clusters or limited execution intervals and does not handle deep learning tasks that require saving and resuming the training state. Consequently, training continuity is not guaranteed for deep learning workloads. Haghshenas et al. proposed an algorithm that determines temporal or spatial scheduling decisions for deep learning training workloads [14]. However, this method relies on an algorithm based on collected carbon intensity data. It does not migrate running training workloads in response to real-time changes in carbon intensity.

Commercial cloud platforms have also introduced tools that enable users to track and monitor carbon emissions associated with their platforms. Microsoft Azure [22], Amazon Web Services [23], and Google Cloud [24] each provide dashboards for monitoring and analyzing both carbon emissions and carbon intensity within their cloud infrastructures. However, these tools focus on usage-based aggregation and report generation. They lack mechanisms to mitigate emissions through workload shifting or migration.

Recently, workload scheduling software that leverages carbon intensity has emerged as a research focus beyond simple usage-based aggregation and monitoring [25–27]. Bahreini et al. proposed a controller that exploits temporal and spatial variations in carbon intensity to minimize emissions [25]. Jiang et al. discussed a carbon-aware scheduler for serverless computing that optimizes carbon emissions while maintaining service time [26]. Chadha et al. proposed a carbon-aware scheduling framework that distributes workloads across time periods and regions with lower carbon intensity [27]. However, these software systems primarily target single or short-term workloads. As a result, their scheduling support has limitations in handling long-term workloads such as deep learning. Moreover, unified multi-cloud resource management and carbon-aware operation of long-term workloads are not supported.

In our previous work, we proposed an approach to reduce the carbon footprint of deep learning workloads in multi-cloud environments. While our previous work established the methodological foundation for carbon-aware migration through experimental validation, GreenAccounter extends these techniques into a practical, deployable, and open-source software toolkit. In particular, GreenAccounter is designed to complete the entire operational lifecycle of measurement, optimization, and reporting within a single platform. It encompasses real-time carbon data measurement, threshold-based automated checkpoint migration for emissions optimization, and an integrated dashboard for visualization and emissions reporting. Additionally, the Kubernetes-based cloud-native architecture provides unified observability across multi-cloud resources and enables the sustainable operation of large-scale deep learning workloads in geographically distributed environments.

3. Software description

This section presents the architecture and core functionalities of GreenAccounter.

3.1. Overview

GreenAccounter is a cloud-based toolkit designed to optimize and monitor carbon emissions from deep learning workloads. A unified dashboard enables users to view and manage operational metrics and event logs across multiple clouds in real-time. The toolkit provides core functions for ensuring training continuity through checkpoint saving and restoration. It also delivers integrated management of cloud metrics and event logs.

To ensure continuity of long-term deep learning workloads, GreenAccounter combines checkpoint-based migration with cloud resource management. When carbon intensity exceeds a threshold or a service failure occurs, the toolkit stores the training state as a checkpoint. It then transfers the checkpoint to another cloud with lower carbon intensity, where training resumes. This design achieves both fault tolerance and continuity of training. Additionally, it allows for the adjustment of training locations according to regional variations in carbon intensity.

GreenAccounter manages deep learning workloads through a three-phase process comprising collection, migration, and monitoring. This process enables carbon-aware scheduling and real-time control. In the collection phase, the system gathers external carbon intensity data. It also collects internal monitoring results such as resource usage, power consumption, and training status of cloud servers. During the migration phase, the system analyzes this information and relocates the workload to a region with lower carbon intensity. When migration occurs, it saves the training state as a checkpoint, transfers it to another cloud, and resumes execution. In the monitoring phase, the dashboard visualizes aggregated results. Key indicators are current carbon intensity, cumulative emissions, and resource usage. To enable emissions tracking, GreenAccounter computes the carbon emissions attributable to the workload in the cloud region. The carbon emissions $E_r(T)$ in cloud region r by time T are defined as:

$$E_r(T) = \sum_{\substack{k=1 \\ r_k=r}}^{K(T)} PC(t_k, r_k) \times CI(t_k, r_k) \times \Delta t. \tag{1}$$

where $K(T)$ denotes the total number of sampling intervals performed up to time T, and k indexes each monitoring interval. $PC(t_k, r_k)$ represents the real-time power consumption of the hardware measured at time t_k in region r_k, while $CI(t_k, r_k)$ denotes the corresponding regional carbon intensity. The condition $r_k = r$ ensures that only the emissions generated while the workload is executing in region r are attributed to that region. Finally, Δt indicates the fixed monitoring interval of the toolkit.

GreenAccounter aggregates the region-specific emissions to calculate the cumulative carbon emissions $E(T)$ across all regions as follows:

$$E(T) = \sum_{k=1}^{K(T)} PC(t_k, r_k) \times CI(t_k, r_k) \times \Delta t. \tag{2}$$

where $E(T)$ denotes the cumulative carbon emissions incurred by the workload up to time T. These indicators enable users to track training progress and emissions in real-time and to refine operational strategies.

3.2. Software architecture and functionalities

As shown in Fig. 1, the overall architecture of GreenAccounter adopts a microservice architecture (MSA). The purpose is to manage functionalities with different operational cycles and scalability requirements, such as monitoring and migration control. This design separates each functionality into an independent service unit, thereby ensuring scalability and fault tolerance. The architecture consists of four layers: user, dashboard, service, and observability. This separation supports user accessibility, flexible service integration, independence of core functionalities, and a stable execution environment.

User layer: This layer provides an entry point for users to access the dashboard and operate their workloads continuously. When a new cloud is connected, the system securely registers authentication information such as API credentials and secure shell (SSH) keys. It then stores log records, including hardware details of the cloud, in the storage.

Dashboard layer: This layer supports integration across functionalities and manages interactions between users and the dashboard. It uses a React-based web dashboard, the Flask framework, and an Nginx server. It also visualizes operational metrics and event logs. Through this layer, users can remotely access cloud instances and manage workloads, including starting and stopping, within a single interface.

Service layer: This layer collects and stores both carbon intensity and cloud resource information, and it performs workload migration based on this data. Its core logic ensures continuity by generating and applying checkpoints, allowing training to resume from the previous state

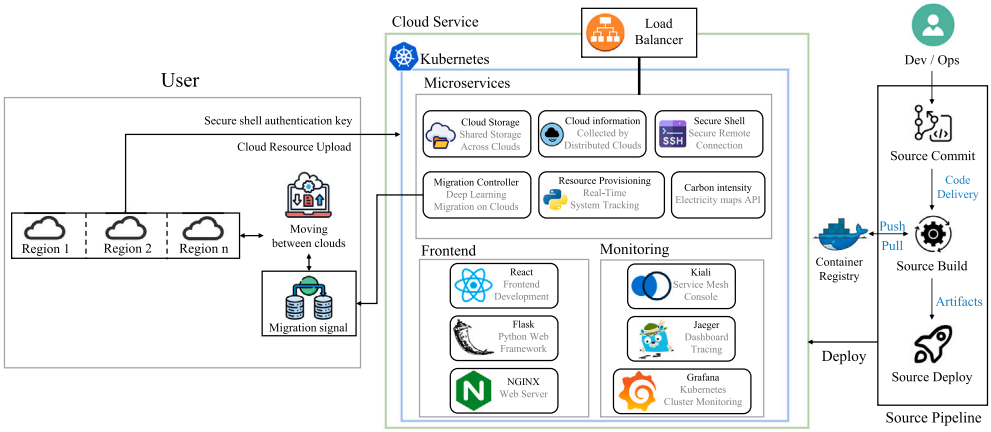

Fig. 1. Overview of the GreenAccounter software architecture and its main components.

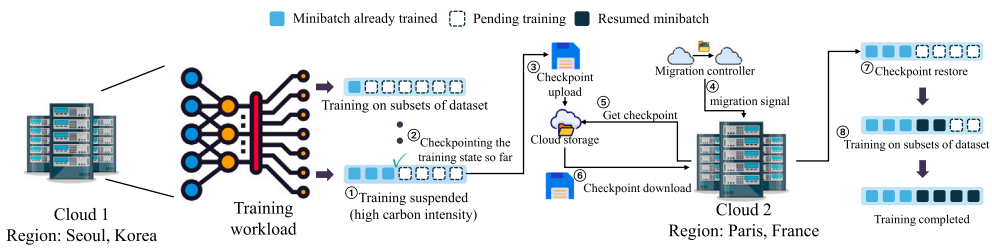

Fig. 2. Migration controller operation for carbon-aware deep learning workloads.

after migration. The Carbon Intensity Collector gathers carbon intensity data for carbon-aware operations. The Cloud Information Collector acquires resource information from connected clouds to provide visibility through the unified dashboard. The cloud storage stores both collected data and checkpoints. The Migration Controller integrates this information and determines whether to migrate workloads to clouds in regions with lower carbon intensity.

Observability layer: This layer ensures system stability and reliability while supporting continuous improvement and rapid deployment. It utilizes observability tools, including Jaeger and Grafana, to monitor service calls, logs, and resource states. This process ensures availability and reliability. The CI/CD pipeline enables independent and rapid deployment at the microservice level. It ensures that functional enhancements or bug fixes are applied reliably and consistently.

GreenAccounter provides a unified dashboard that enables real-time monitoring and control of workload progress, resource utilization, carbon emissions, and migration status across multiple clouds. The system comprises six microservices that interact through collection, migration, and monitoring processes. The unified dashboard integrates them to support multi-cloud management and real-time carbon footprint monitoring. Their roles are summarized as follows:

- **Cloud storage:** Stores operational metrics, logs, checkpoints, time-series carbon intensity, and resource utilization. It provides replication and versioning to ensure recovery without data loss.
- **Cloud information collector:** Collects hardware usage, power consumption, and regional information from connected clouds. It uses a queue-based streaming method to handle data efficiently.
- **Secure shell connection:** Enables remote access to instances for command execution and file transfer through the dashboard.
- **Migration controller:** Determines whether to migrate workloads by analyzing carbon intensity, resource availability, and checkpoints. It triggers checkpoint storage, transfer, and restoration.
- **Resource provisioning:** Allocates and scales cloud resources using Kubernetes templates and policies. It replaces failed nodes to maintain continuous operation.
- **Carbon intensity collector:** Retrieves real-time carbon intensity data from electricity maps and supports emissions estimation and migration decisions.

4. Illustrative examples

This section presents how carbon-aware migration of deep learning workloads operates in practice. It then introduces the main layout of the GreenAccounter dashboard.

4.1. Carbon-aware migration of deep learning workloads

As shown in Fig. 2, the GreenAccounter controller manages the migration of carbon-aware deep learning workloads. The process coordinates checkpoint storage, transfer, and restoration to ensure training continuity when the controller migrates workloads to another region.

Migration decisions in GreenAccounter follow a threshold-based greedy migration strategy. This process is initiated when the carbon intensity of the current cloud region exceeds a predefined threshold. Let t be the current time, r be a cloud region, and R be the set of all connected candidate cloud regions. We define the migration decision function $Trigger(t)$ as follows:

$$Trigger(t) = \begin{cases} 1, & \text{if } CI(t, r_{curr}) > \theta_{CI}, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

where r_{curr} denotes the current execution region of the workload, and θ_{CI} denotes the user-configured carbon intensity threshold. A value of $Trigger(t) = 1$ indicates that a migration event is triggered.

Based on the $Trigger(t)$ signal, the migration controller suspends the current workload. It then generates a checkpoint file that captures the complete training state, including model parameters and optimizer

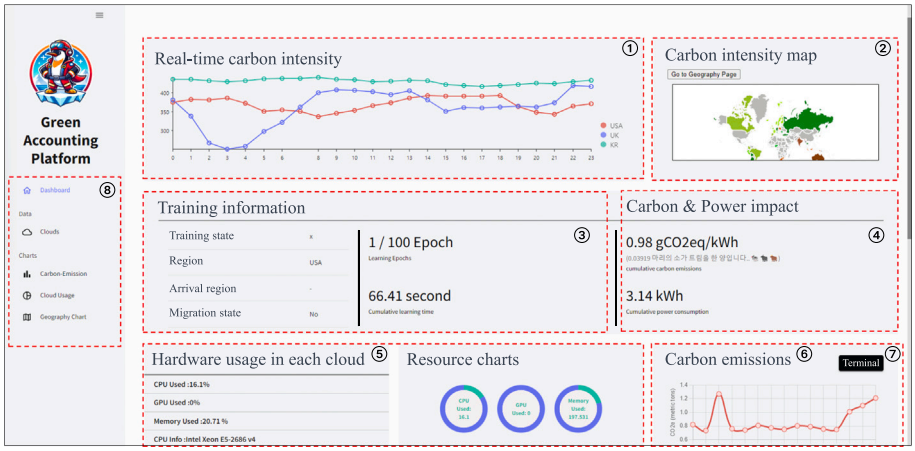

Fig. 3. The screenshot of the GreenAccounter dashboard.

states, and uploads it to cloud storage. Simultaneously, the controller evaluates the real-time carbon intensity of all candidate regions in R to identify the optimal target region r^* as follows:

$$r^* = \arg\min_{r \in R} CI(t, r). \tag{4}$$

where $CI(t, r)$ denotes the carbon intensity measured in region r at time t. Finally, the target cloud r^* downloads the checkpoint from storage, restores the training state, and resumes execution from the precise point of interruption.

Training continuity requires identical execution environments across clouds, which obliges the target region to reproduce the same configuration. GreenAccounter achieves this requirement through a Docker container–based deployment. The target cloud retrieves the environment image from the container registry and reproduces the same configuration. This approach enables workload migration that maintains training continuity while sustaining operations in carbon-efficient cloud environments.

4.2. Dashboard application

Fig. 3 presents a screenshot of the dashboard, which consists of a main layout and eight panels. The GreenAccounter dashboard provides a unified interface for monitoring deep learning workloads. Table 1 summarizes the functionality of each panel, all of which collectively deliver key metrics for workload execution and cloud operations.

The dashboard classifies workload information into three categories to ensure balanced visibility. The monitoring indicators report regional carbon intensity, training progress, and migration status in real-time. Environmental impact indicators measure cumulative carbon emissions and total power consumption. Resource utilization indicators visualize CPU, GPU, and memory usage across clouds. The dashboard also supports direct control of connected instances through a remote terminal within the same interface.

Specific panels highlight both the environmental benefits of migration and system efficiency. Carbon intensity and cumulative emissions indicators (Panels 1, 2, 4, and 6) demonstrate reductions in carbon output due to migration. Training progress and resource utilization indicators (Panels 3 and 5) provide insight into both performance and efficiency. This design enables users to establish operational strategies that balance carbon reduction with resource optimization across multi-cloud environments.

5. Impact

GreenAccounter broadens the traditional focus on cost and performance optimization to include carbon-aware workload placement and migration. Prior studies addressed short-term workloads or static deployment. In contrast, GreenAccounter adapts to real-time variations in carbon intensity and ensures continuity through checkpoint-based migration. This approach extends the core techniques of the previous research [8] into a practical platform. It enables future researchers to investigate new scheduling problems that simultaneously optimize latency, cost, and carbon reduction.

Although workload migration entails data transfer overheads, prior studies indicate that these costs are negligible relative to the carbon benefits [8,28,29]. Recent studies estimate the carbon intensity of data transmission to range from 1.51 g [28] to 11 g [29] of CO2 per GB. Consequently, transferring even a substantial 100 GB checkpoint generates approximately 0.15–1.1 kg of CO2, a figure significantly lower than the operational carbon savings achieved through migration. Furthermore, our prior empirical analysis demonstrated that transfer latency is minimal, leveraging the high-performance network infrastructure of modern data centers [8]. These findings confirm that the proposed approach yields a net positive environmental impact without compromising training continuity.

Table 1
Description of dashboard panels and their functions.

No.	Panel name	Description
1	Real-time carbon intensity	Real-time carbon intensity trends are displayed as time-series graphs for each connected cloud region.
2	Carbon intensity map	A global map visualizes regional carbon intensity through color-coded distribution.
3	Training information	Training status, progress, cumulative training time, and migration event logs are provided for workload monitoring.
4	Carbon & Power impact	Cumulative carbon emissions and total power consumption are reported as numerical indicators. Their corresponding environmental impact is also provided.
5	Hardware usage in each cloud	Resource utilization of CPU, GPU, and memory is visualized along with hardware specifications for each cloud.
6	Carbon emissions	Real-time carbon emissions of connected clouds are presented as time-series graphs.
7	Terminal connection	A terminal connection button enables remote control of each cloud instance from the dashboard.
8	Navigation panel	A navigation panel provides access to reports on cloud status, carbon emissions, and resource usage.

The proposed toolkit improves accessibility for research and experimentation. The toolkit is open-source software that provides an environment for researchers to benchmark diverse algorithms in carbon-aware workload scheduling. This availability supports interdisciplinary applications across computer science and sustainability research. It also serves as a practical tool for data center operators. These operators aim to establish strategies that reduce their carbon footprints.

6. Conclusions and future work

GreenAccounter is an open-source toolkit that implements checkpoint-based migration for carbon-aware deep learning workloads. It ensures continuity during long-term training and demonstrates that workloads can adapt to changes in carbon intensity while minimizing emissions. The toolkit provides a standardized experimental environment for researchers to evaluate and compare scheduling strategies. It offers operators a practical tool to monitor and reduce carbon emissions in multi-cloud environments. Its core methodology builds on prior experimental research [8] and extends it into a deployable toolkit for workload management.

Future extensions will include support for diverse workloads, integration of cost models, and GPU frequency optimization to achieve more precise monitoring of emissions.

CRediT authorship contribution statement

Jeonghyeon Park: Writing – original draft, Validation, Supervision, Software, Project administration, Methodology. **Jaekyeong Kim:** Software, Project administration. **Wonseok Son:** Software, Project administration. **Sejin Chun:** Writing – review & editing, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Dong-A University research fund.

References

[1] Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al. Gpt-4 technical report. [arXiv preprint] arXiv:2303.08774. 2023.
[2] Gupta U, Kim YG, Lee S, Tse J, Lee H-HS, Wei G-Y, Brooks D, Wu C-J. Chasing carbon: the elusive environmental footprint of computing. In: 2021 IEEE international symposium on High-Performance computer architecture (HPCA); 2021. p. 854–67.
[3] Patterson D, Gonzalez J, Hölzle U, Le Q, Liang C, Munguia L-M, Rothchild D, So DR, Texier M, Dean J. The carbon footprint of machine learning training will plateau, then shrink. Computer 2022;55(7):18–28.
[4] Dodge J, Prewitt T, Tachet des Combes R, Odmark E, Schwartz R, Strubell E, Luccioni AS, Smith NA, DeCario N, Buchanan W. Measuring the carbon intensity of AI in cloud instances. In: Proceedings of the 2022 ACM conference on fairness, accountability, and transparency; 2022. p. 1877–94.
[5] Zhu B, Deng Z, Song X, Zhao W, Huo D, Sun T, Ke P, Cui D, Lu C, Zhong H, et al. Carbonmonitor-power near-real-time monitoring of global power generation on hourly to daily scales. Scientific Data 2023;10(1):217.
[6] Bian J, Wang L, Ren S, Xu J. Cafe: carbon-aware federated learning in geographically distributed data centers. In: Proceedings of the 15th ACM international conference on future and sustainable energy systems; 2024. p. 347–60.
[7] Xu W, Huang X, Meng S, Zhang W, Guo L, Sato K. An efficient checkpointing system for large machine learning model training. In: Sc24-w: workshops of the international conference for high performance computing, networking, storage and analysis; 2024. p. 896–900.
[8] Park J, Kim D, Kim J, Han J, Chun S. Carbon-aware and fault-tolerant migration of deep learning workloads in the geo-distributed cloud. In: 2024 IEEE 17th international conference on cloud computing (CLOUD); 2024. p. 494–501.
[9] Khodayarseresht E, Shameli-Sendi A, Fournier Q, Dagenais M. Energy and carbon-aware initial VM placement in geographically distributed cloud data centers. Sustain Comput Inform Syst 2023;39:100888.
[10] Radovanović A, Koningstein R, Schneider I, Chen B, Duarte A, Roy B, Xiao D, Haridasan M, Hung P, Care N, et al. Carbon-aware computing for datacenters. IEEE Trans Power Syst 2022;38(2):1270–80.
[11] Bahreini T, Tantawi A, Youssef A. An approximation algorithm for minimizing the cloud carbon footprint through workload scheduling. In: 2022 IEEE 15th international conference on cloud computing (CLOUD); 2022. p. 522–31.
[12] Tripathi S, Kumar P, Gupta P, Misra R, Singh TN. Workload shifting based on low carbon intensity periods: a framework for reducing carbon emissions in cloud computing. In: 2023 IEEE international conference on big data (BigData); 2023. p. 3387–95.
[13] Nabavinejad SM, Jia W, Taneja S, Guo T. Carbondis: carbon-aware DNN inference scheduling on heterogeneous gpus. In: 2025 IEEE 25th international symposium on cluster, cloud and internet computing (CCGrid); 2025. p. 315–24.
[14] Haghshenas K, Setz B, Aiello M. CO 2 emission aware scheduling for deep neural network training workloads. In: 2022 IEEE international conference on big data (big data); 2022. p. 1542–9.
[15] Piontek T, Haghshenas K, Aiello M. Carbon emission-aware job scheduling for kubernetes deployments. J Supercomput 2023;80:549–69.
[16] Sukprasert T, Souza A, Bashir N, Irwin DE, Shenoy PJ. Quantifying the benefits of carbon-aware temporal and spatial workload shifting in the cloud. arXiv preprint arXiv:2306.06502. 2023.
[17] Su S, Zhou Z, Ouyang T, Zhou R, Chen X. Learning to be green: carbon-aware online control for edge intelligence with colocated learning and inference. In: 2023 IEEE 43rd international conference on distributed computing systems (ICDCS); 2023. p. 567–78.
[18] Bahreini T, Tantawi A, Youssef A. A carbon-aware workload dispatcher in cloud computing systems. In: 2023 IEEE 16th international conference on cloud computing (CLOUD); 2023. p. 212–8.
[19] Saad Z, Yang J, Leung H, Drew S. Towards carbon-aware container orchestration: Predicting workload energy consumption with federated learning. [arXiv preprint] arXiv:2510.03970. 2025.
[20] Raith P, Rattihalli G, Dhakal A, Chalamalasetti SR, Milojicic D, Frachtenberg E, Nastic S, Dustdar S. Opportunistic energy-aware scheduling for container orchestration platforms using graph neural networks. In: 2024 IEEE 24th international symposium on cluster, cloud and internet computing (CCGrid); 2024. p. 299–306.
[21] Sukprasert T, Souza A, Bashir N, Irwin D, Shenoy P. On the limitations of carbon-aware temporal and spatial workload shifting in the cloud. In: Proceedings of the nineteenth european conference on computer systems; 2024. p. 924–41.
[22] Microsoft Corporation. Emissions impact dashboard. https://www.microsoft.com/en-us/sustainability/emissions-impact-dashboard [accessed: 6 February 2026].
[23] Amazon Web Services, Inc. AWS customer carbon footprint tool. https://aws.amazon.com/ko/aws-cost-management/aws-customer-carbon-footprint-tool/ [accessed: 6 February 2026].
[24] Google LLC. Cloud carbon footprint. https://cloud.google.com/carbon-footprint [accessed: 6 February 2026].
[25] Bahreini T, Tantawi AN, Tardieu O. Caspian: a carbon-aware workload scheduler in multi-cluster kubernetes environments. In: 2024 32nd international conference on modeling, analysis and simulation of computer and telecommunication systems (MASCOTS); 2024. p. 1–8.
[26] Jiang Y, Roy RB, Li B, Tiwari D. Ecolife: carbon-aware serverless function scheduling for sustainable computing. In: Sc24: international conference for high performance computing, networking, storage and analysis; 2024. p. 1–15.
[27] Chadha M, Subramanian T, Arima E, Gerndt M, Schulz M, Abboud O. Greencourier: carbon-aware scheduling for serverless functions. In: Proceedings of the 9th international workshop on serverless computing; 2023. p. 18–23.
[28] Mersy G, Krishnan S. Toward a life cycle assessment for the carbon footprint of data. ACM SIGENERGY Energy Inform Rev 2024;4(5):25–33.
[29] Aslan J, Mayers K, Koomey JG, France C. Electricity intensity of internet data transmission: untangling the estimates. J Ind Ecol 2018;22(4):785–98.